

...

8.21.ipynb

🔗 hjälpfunktioner.py

🔗 tesfil.py

🔗 schrödinger.py X

🔗 schrödingeruppgiftc.py

🔗 schrödingestjärna.py

🖼️ stjärn_konvergens

Flervarre > 🔗 schrödinger.py > ...

```
1 import numpy as np
2 from scipy.sparse.linalg import eigsh
3 import matplotlib.pyplot as plt
4 from hjälpfunktioner import generate_mesh_2d, assemble_matrix_2d, compute_element_matrix_2d
5 #Steg 1: Skapa nätet
6
7 #generate_mesh_2d(32,32) skapar (32+1)*(32+1) = 1089 noder
8 L = 1
9 nx, ny = 32, 32
10 V, K = generate_mesh_2d(nx, ny)
11
12 #Steg 2: Definiera lhs-funktioner
13 #Styvhetismatris A:  $(1/2) * \text{grad}_u \cdot \text{grad}_v$ 
14 def schrodinger_lhs(u, v, grad_u, grad_v, x, dx):
15     return 0.5 * np.dot(grad_u, grad_v) * dx
16
17 # Massmatris M:  $u * v$  använder bara funktionsvärden, ingen gradient
18 #vi måste ha denna då lösningen av  $Au = \lambda u$  antar att integralen av  $u * v = 1$  vilket inte är sant
19 #så vi måste istället lösa  $Au = \lambda M * u$  vilket är det vi gör sen.
20 def mass_lhs(u, v, grad_u, grad_v, x, dx):
21     return u * v * dx
22
23 # Steg 3: Assemblera matriserna
24 # assemble_matrix_2d loopar över alla element K och
25 # anropar compute_element_matrix_2d för varje triangel
26 A = assemble_matrix_2d(schrodinger_lhs, V, K)
27 M = assemble_matrix_2d(mass_lhs, V, K)
28
29 # Steg 4: Hitta inre noder (ej på randen)
30 # Randnoder har x=0, x=1, y=0 eller y=1
31 # Vi tar bara med inre noder i egenvärdesproblemet, det är det denna for loop hittar
32 eps = 1e-10
33 interior = []
34 for i in range(len(V)):
35     v = V[i]
36     if v[0] > eps and v[0] < 1-eps and v[1] > eps and v[1] < 1-eps:
37         interior.append(i)
38
39 # Steg 5: Extrahera sub-matriser för inre noder
```

```

37 # Steg 5: Extrahera sub-matriser för inre noder
38
39 # Vi plockar ut raderna och kolumnerna som svarar mot inre noder
40 # Detta är ekvivalent med att sätta u=0 på randen
41 A_int = A[interior, :][:, interior].tocsr() #styvhetsmatrisen för endast inre noder
42 M_int = M[interior, :][:, interior].tocsr() #Massmatrisen men bara för inre noder
43 #.tocsr gör den till en sparse-format om vi kan lösa egenvärdesproblemet med
44
45 # Steg 6: Lös generaliserat egenvärdesproblem
46 # eigsh hittar de k minsta egenvärden för  $A*u = \lambda*M*u$ 
47 # which='SM' = Smallest Magnitude
48 num_eigs = 4
49 eigenvalues, eigenvectors = eigsh(A_int, k=num_eigs, M=M_int, which='SM')
50
51 # Sortera efter storleksordning (minst först)
52 idx = np.argsort(eigenvalues)
53 eigenvalues = eigenvalues[idx]
54 eigenvectors = eigenvectors[:, idx]
55
56 # Mappa tillbaka till alla noder (randnoder = 0)
57 N = len(V)
58 U_full = np.zeros((N, num_eigs))
59 for k in range(num_eigs):
60     U_full[interior, k] = eigenvectors[:, k]
61
62 # Steg 7: Plotta egenfunktionerna med tricontourf
63
64 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
65 fig.suptitle('De fyra lägsta egenfunktionerna (L=1)', fontsize=14)
66
67 for k in range(num_eigs):
68     ax = axes[k // 2, k % 2]
69     # tricontourf(x-kord, y-kord, trianglar, värden)
70     tcf = ax.tricontourf(V[:, 0], V[:, 1], K, U_full[:, k], levels=20, cmap='RdBu_r')
71     fig.colorbar(tcf, ax=ax)
72     ax.set_title(f' $\lambda_{k+1} = {eigenvalues[k]:.4f}$ ') #sätt titel för varje graf
73     ax.set_xlabel('x1') #vilket axel det visar

```

...

8.21.ipynb

hjälpfunktioner.py

tesfil.py

schrödinger.py X

schrödingeruppgiftc.py

schrödingestjärna.p

Flervarre > schrödinger.py > ...

```
73     ax.set_xlabel('x1') #vilket axel det visar
74     ax.set_ylabel('x2')
75
76     plt.tight_layout()
77     plt.savefig('egenfunktioner.png', dpi=150)
78     plt.show()
79
80     # Jämför med analytiska egenvärden
81
82     print("Numeriska egenvärden:") #vilka värden vi får när vi räknar ut med python
83     for k in range(num_eigs):
84         print(f"    lambda_{k+1} = {eigenvalues[k]:.6f}")
85
86     print("\nAnalytiska egenvärden lambda_mn = (n^2+m^2)*pi^2/2:")
87
88     analytiska = [] #vilka värden vi fick på papper i tidigare uppgiften.
89     for n in range(1, 4):
90         for m in range(1, 4):
91             lam = (n**2 + m**2) * np.pi**2 / 2
92             analytiska.append(lam)
93     analytiska = sorted(analytiska)
94
95     for k in range(len(analytiska[:6])):
96         lam = analytiska[k]
97         print(f"    lambda_{k+1} = {lam:.6f}")
```

... 8.21.ipynb 📄 hjälpfunktioner.py 📄 tesfil.py 📄 schrödinger.py 📄 schrödingeruppgiftc.py ✕

📄 schrödingestjärna.py

🖼️ stjärn_konvergens.png

▶ ▾ 📄 ...

Flervarre > 📄 schrödingeruppgiftc.py > ...

```
1 import numpy as np
2 from scipy.sparse.linalg import eigsh
3 import matplotlib.pyplot as plt
4 from hjälpfunktioner import generate_mesh_2d, assemble_matrix_2d
5 #hela detta programmet fungerar egentligen typ exakt samma som det första bara att vi loopad igenom de olika näten
6 #samt att vi sedan beräknas konvergensordningen samt plottar den på en log log graf
7 #och jämför med en refsenslinje vilket är den linjen som den vi beräkna bör följa.
8
9 #Steg 1: Definiera lhs-funktioner (samma som i (b))
10 def schrodinger_lhs(u, v, grad_u, grad_v, x, dx):
11     return 0.5 * np.dot(grad_u, grad_v) * dx
12
13 def mass_lhs(u, v, grad_u, grad_v, x, dx):
14     return u * v * dx
15
16 #Steg 2: Analytiskt egenvärde att jämföra mot
17 lambda_exact = np.pi**2 #lambda_11 = (1^2+1^2)pi^2/2 = pi^2
18
19 #Steg 3: Loopa över nätsstorlekar
20
21 #n_values är antalet element i varje riktning
22 #h = 1/n är steglängden
23 n_values = [8, 16, 32, 64, 128]
24 h_values = []
25 errors = []
26
```

Flervarre > schrödingeruppgiftc.py > ...

```
20
27 for n in n_values:
28     print(f"Beräknar för {n}x{n}-nät:")
29     #Skapa nätet för aktuella n_values
30     h = 1 / n
31     V, K = generate_mesh_2d(n, n)
32     #Assemblera matriserna för aktuellt nät
33     A = assemble_matrix_2d(schrodinger_lhs, V, K)
34     M = assemble_matrix_2d(mass_lhs, V, K)
35
36     #Hitta inre noder (samma som i (b))
37     eps = 1e-10
38     interior = []
39     for i in range(len((V))):
40         v = V[i]
41         if v[0] > eps and v[0] < 1-eps and v[1] > eps and v[1] < 1-eps:
42             interior.append(i)
43
44     #Extrahera sub-matriser för inre noder ( samma som i (b))
45     A_int = A[interior, :][:, interior].tocsr()
46     M_int = M[interior, :][:, interior].tocsr()
47
48     #Beräkna minsta egenvärdet
49     #k=1 eftersom vi bara behöver det minsta, det är det som vi ska jämföra med sen.
50     eigenvalues, _ = eigsh(A_int, k=1, M=M_int, which='SM')
51     lambda_h = eigenvalues[0]
52
53     #Beräkna felet
54     error = abs(lambda_h - lambda_exact)
55
56     #Spara h och felet
57     h_values.append(h)
58     errors.append(error)
59
60     print(f" h = {h:.4f}, lambda_h = {lambda_h:.6f}, fel = {error:.2e}")
61
62 #Steg 4: Beräkna konvergensordningen
63 #Tänk på att konvergensfelet vanligtvis följer  $\lambda_h - \lambda_{\text{exact}} \propto h^2$ 
```

```

61
62 #Steg 4: Beräkna konvergensordningen
63 #I en log-log plot ska felet vara en rät linje:  $\log(\text{errors}) = p \cdot \log(h) + C$ 
64 #Vi skattar lutningen p med numpy.polyfit
65 log_h = np.log(h_values)
66 log_e = np.log(errors)
67
68 #polyfit(x, y, 1) passar en rät linje till datan
69 #koefficienter[0] är lutningen = konvergensordningen p
70 koefficienter = np.polyfit(log_h, log_e, 1)
71 p = koefficienter[0]
72 print(f"\nKonvergensordning p = {p:.2f}")
73
74 #Steg 5: Plotta log-log
75
76 fig, ax = plt.subplots(figsize=(7, 5))
77 #Plotta felet
78 ax.loglog(h_values, errors, 'bo-', label='FEM-fel  $\lambda_h - \lambda_{11}$ ')
79
80 #Plotta en referenslinje med lutning p för jämförelse
81 #Vi skalar den så att den passar vår data
82 h_ref = np.array(h_values)
83 ref_line = errors[0] * (h_ref / h_values[0])**p
84 ax.loglog(h_ref, ref_line, 'r--', label=f'Referens  $h^{{p:.1f}}$ ') #referenslinje
85
86 ax.set_xlabel('h: (nätstorlek)', fontsize=12)
87 ax.set_ylabel('Fel:  $\lambda_h - \lambda_{11}$ ', fontsize=12)
88 ax.set_title('Konvergens för minsta egenvärdet', fontsize=13)
89 ax.legend(fontsize=11)
90 ax.grid(True, which='both', linestyle='--', alpha=0.5)
91 plt.tight_layout()
92 plt.savefig('konvergens.png', dpi=150)
93 plt.show()

```

Flervarre > schrodingerstjärna.py > ...

```
1 import numpy as np
2 from scipy.sparse.linalg import eigsh
3 import matplotlib.pyplot as plt
4 from hjälpfunktioner import generate_mesh_2d, assemble_matrix_2d
5 #nu ska vi upprepda exakt samma algoritm egentligen men nu finns en ny del i ekvationen eftersom  $V = 10^3(x_1 - 1/2)^2$ 
6 #detta gör att vi får ännu en lhs funktion att beräkna och denna kallas potential.
7 #denna ser nästan ut som massmatrisen som multiplicerar funktionsvärdena men här också med V alltså
8 #integralen över  $V(x_1, x_2) * v * u * dx$  i svag form vilket gör att vi får den nya potential funktionen nedan.
9
10 #fulltändiga ekvationen på svag form är alltså:
11 #integral av  $1/2 \text{ grad}_U * \text{grad}_v * dx$  + integral av  $V(x_1, x_2) * u * v * dx = \text{lambda} * \text{integral av } u * v * dx$ 
12
13 #Steg 1: Definiera lhs-funktioner
14 #Kinetisk del, samma som tidigare
15 def kinetic_lhs(u, v, grad_u, grad_v, x, dx):
16     return 0.5 * np.dot(grad_u, grad_v) * dx
17
18 #Potentialdel  $V(x_1, x_2) * u * v$ , V utvärderas i kvadraturpunkten x
19 def potential_lhs(u, v, grad_u, grad_v, x, dx):
20     V = 1e3 * (x[0] - 0.5)**2 # $V(x_1, x_2) = 10^3 * (x_1 - 0.5)^2$ 
21     return V * u * v * dx #ingen gradient, som massmatrisen
22
23 #Massmatris, samma som tidigare
24 def mass_lhs(u, v, grad_u, grad_v, x, dx):
25     return u * v * dx
26 # Steg 2: Assemblera  $A = A_{\text{kinetisk}} + A_{\text{potential}}$ 
27
```

```

Flervarre > schrödingestjärna.py > ...
24 def mass_lhs(u, v, grad_u, grad_v, x, dx):
25     return u * v * dx
26 # Steg 2: Assemblera A = A_kinetisk + A_potential
27
28 def bygg_och_los(n):
29     V_noder, K = generate_mesh_2d(n, n)
30
31     # Assemblera de två bidragen till A separat och addera
32     A_kin = assemble_matrix_2d(kinetic_lhs, V_noder, K)
33     A_pot = assemble_matrix_2d(potential_lhs, V_noder, K)
34     A = A_kin + A_pot
35
36     M = assemble_matrix_2d(mass_lhs, V_noder, K)
37     #Hitta inre noder (samma som tidigare)
38     eps = 1e-10
39     interior = []
40     for i in range(len(V_noder)):
41         v = V_noder[i]
42         if v[0] > eps and v[0] < 1-eps and v[1] > eps and v[1] < 1-eps:
43             interior.append(i)
44
45     #Extrahera sub-matriser ( samma som tidigare)
46     A_int = A[interior, :][:, interior].tocsr()
47     M_int = M[interior, :][:, interior].tocsr()
48
49     return V_noder, K, interior, A_int, M_int
50
51 # Steg 3: Plotta 4 lägsta egenfunktionerna (32x32-nät)
52
53 n = 32
54 V_noder, K, interior, A_int, M_int = bygg_och_los(n)
55 num_eigs = 4
56 eigenvalues, eigenvectors = eigsh(A_int, k=num_eigs, M=M_int, which='SM')
57

```

```

57
58 # Sortera minst först
59 idx = np.argsort(eigenvalues)
60 eigenvalues = eigenvalues[idx]
61 eigenvectors = eigenvectors[:, idx]
62
63 #Mappa tillbaka till alla noder (samma som i (a))
64 N = len(V_noder)
65 U_full = np.zeros((N, num_eigs))
66 for k in range(num_eigs):
67     U_full[interior, k] = eigenvectors[:, k]
68
69 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
70 fig.suptitle('Fyra lägsta egenfunktioner med  $V = 10^3(x_1 - 1/2)^2$ ', fontsize=13)
71
72 #samma for loop som i (a)
73 for k in range(num_eigs):
74     ax = axes[k // 2, k % 2]
75     tcf = ax.tricontourf(V_noder[:, 0], V_noder[:, 1], K, U_full[:, k], levels=20, cmap='RdBu_r')
76     fig.colorbar(tcf, ax=ax)
77     ax.set_title(f' $\lambda_{k+1} = \{eigenvalues[k]:.4f\}$ ')
78     ax.set_xlabel('x_1')
79     ax.set_ylabel('x_2')
80
81 plt.tight_layout()
82 plt.savefig('stjärn_egenfunktioner.png', dpi=150)
83 plt.show()
84
85 #Steg 4:Konvergensstudie - jämför mot finaste nätet
86
87 #Ingen analytisk lösning finns, så vi använder 128x128 som referens
88 print("Beräknar referensvärde på 128x128-nät:")
89 _, _, interior_ref, A_ref, M_ref = bygg_och_los(128)
90 lambda_ref, _ = eigsh(A_ref, k=1, M=M_ref, which='SM') #beräkna egenvärden på samma sätt som tidigare, k=1 pga behöver bara det minsta
91 lambda_ref = lambda_ref[0] #ta detta som referensvärde
92 print(f"Referensvärde lambda_ref = {lambda_ref:.8f}")
93

```

```

93
94 # Loopa över grövre nät
95 n_values = [8, 16, 32, 64]
96 h_values = []
97 errors = []
98
99 for n in n_values: #loopa över nät på samma sätt som i uppgift (b) och ska ska vi sedan jämföra dessa grövre med referensen.
100     print(f"Beräknar för {n}x{n}-nät:")
101     _, _, interior_n, A_n, M_n = bygg_och_los(n)
102     lam, _ = eigsh(A_n, k=1, M=M_n, which='SM') #återigen bara minsta som behövs
103     error = abs(lam[0] - lambda_ref) #beräknar error med hjälp av vårt referensvärde
104     h_values.append(1/n)
105     errors.append(error) #spara de olika errorsen för de olika näten.
106     print(f" h = {1/n:.4f}, lambda_h = {lam[0]:.6f}, fel = {error:.2e}")
107
108 # Konvergensordning (samma sätt som i uppgift (b))
109 log_h = np.log(h_values)
110 log_e = np.log(errors)
111 p = np.polyfit(log_h, log_e, 1)[0]
112 print(f"\nKonvergensordning p = {p:.2f}")
113
114 # Plotta
115 fig, ax = plt.subplots(figsize=(7, 5))
116 ax.loglog(h_values, errors, 'bo-', label='FEM-fel lambda_h - lambda_ref')
117 h_ref = np.array(h_values)
118 ref_line = errors[0] * (h_ref / h_values[0])**p
119 ax.loglog(h_ref, ref_line, 'r--', label=f'Referens h^{p:.1f}')
120 ax.set_xlabel('h (nätstorlek)', fontsize=12)
121 ax.set_ylabel('Fel', fontsize=12)
122 ax.set_title('Konvergens med  $V = 10^3(x_1-1/2)^2$ ', fontsize=13)
123 ax.legend(fontsize=11)
124 ax.grid(True, which='both', linestyle='--', alpha=0.5)
125 plt.tight_layout()
126 plt.savefig('stjärn_konvergens.png', dpi=150)
127 plt.show()

```